

Module 4: Calculus Intuition

a. Functions and Rates of Change

- *Basic:* Write `average_rate_of_change(f, x1, x2)` from scratch.
- *Hard:* Using that function, show that the average rate of change of $f(x) = x^2$ gets closer and closer to the true derivative ($2x$) as x_1 and x_2 get closer together.

b. Slopes

- *Basic:* Write `slope(x1, y1, x2, y2)` from scratch.
- *Hard:* Write a function that takes a list of (x,y) points and returns the slope between every consecutive pair — useful for spotting where a curve flattens.

c. Derivatives

- *Basic:* Write `derivative_at_point(f, x, h)` from scratch (numerical approximation).
- *Hard:* Use your derivative function to approximate the derivative of $f(x) = x^3$ at 3 different points, and verify each result matches the true formula $3x^2$.

d. Optimization Intuition

- *Basic:* Write `find_minimum_by_search(f, start, end, step)` from scratch.
- *Hard:* Test your search function on a function with two dips (e.g., $f(x) = (x^2 - 4)^2$) and check whether it finds the true global minimum or gets fooled by a local one.

e. Why Calculus Matters in ML

- *Basic:* Write one full gradient-descent loop (predict $\rightarrow$ derivative $\rightarrow$ step) for $f(x) = (x-5)^2$, starting from $x=0$.
 - *Hard:* Run the same loop with three different learning rates (0.01 , 0.1 , 0.9) and explain what happens differently in each case.
-

Module 5: Optimization Fundamentals

a. Cost Functions

- *Basic:* Write `mean_squared_error(actual, predicted)` from scratch.
- *Hard:* Write `mean_absolute_error(actual, predicted)` too, then compare both on a dataset with one big outlier error — which one is affected more, and why?

b. Error Minimization

- *Basic:* Write `has_improved(old_cost, new_cost)` returning True/False.
- *Hard:* Write a loop that tracks cost over 10 training steps and stops early ("early stopping") if the cost stops improving for 3 steps in a row.

c. Gradient Intuition

- *Basic:* Write `gradient_at_point(cost_fn, x, h)` from scratch.

- *Hard:* For `cost_fn(x) = (x-3)**2`, compute the gradient at `x=0`, `x=3`, and `x=6` — explain why the signs and values differ the way they do.

d. Optimization Workflows

- *Basic:* Write `gradient_descent_step(x, learning_rate, gradient)` from scratch.
- *Hard:* Build a full gradient descent loop for `cost_fn(x) = (x-10)**2` running 50 steps, and print the cost every 10 steps to show convergence.

e. Learning Through Iteration

- *Basic:* Modify your gradient descent loop to count how many steps it takes to get within `0.01` of the true minimum.
 - *Hard:* Compare how many steps it takes to converge using `learning_rate=0.5` vs `learning_rate=0.01` — which is faster, and does either one fail to converge properly?
-

Module 6: ML Mathematics in Practice

a. Linear Regression Intuition

- *Basic:* Write `predict(X_row, weights, bias)` from scratch.
- *Hard:* Given a small dataset of `X` (features) and `y` (labels) and a fixed `weights/bias`, compute MSE across all predictions using functions from Module 5.

b. Similarity Calculations

- *Basic:* Reuse `cosine_similarity` to compare a query vector against 2 document vectors.
- *Hard:* Given 5 document vectors and 1 query vector, rank all 5 documents from most to least similar without hardcoding the order.

c. Distance Measurements

- *Basic:* Write `euclidean_distance(v1, v2)` and `manhattan_distance(v1, v2)` from scratch.
- *Hard:* Given one query point and 4 candidate points, write a simple "nearest neighbor" function that returns the closest candidate using Euclidean distance.

d. Feature Transformations

- *Basic:* Reuse `normalize_column` on a list of ages.
- *Hard:* Write `normalize_matrix(matrix)` that normalizes every column of a full dataset, and show what happens to a KNN-style distance calculation before vs after normalizing two very differently-scaled features.

e. Mathematical Reasoning in ML

- *Basic:* For a "predict exam score from study hours" problem, state which module's concepts apply and why.

- *Hard:* Design (in words, no code) a full mini ML pipeline for a spam classifier — list every mathematical concept from Modules 1-6 involved, in the order they'd be used.